Fruit Ripeness Detection System

Project Engineering

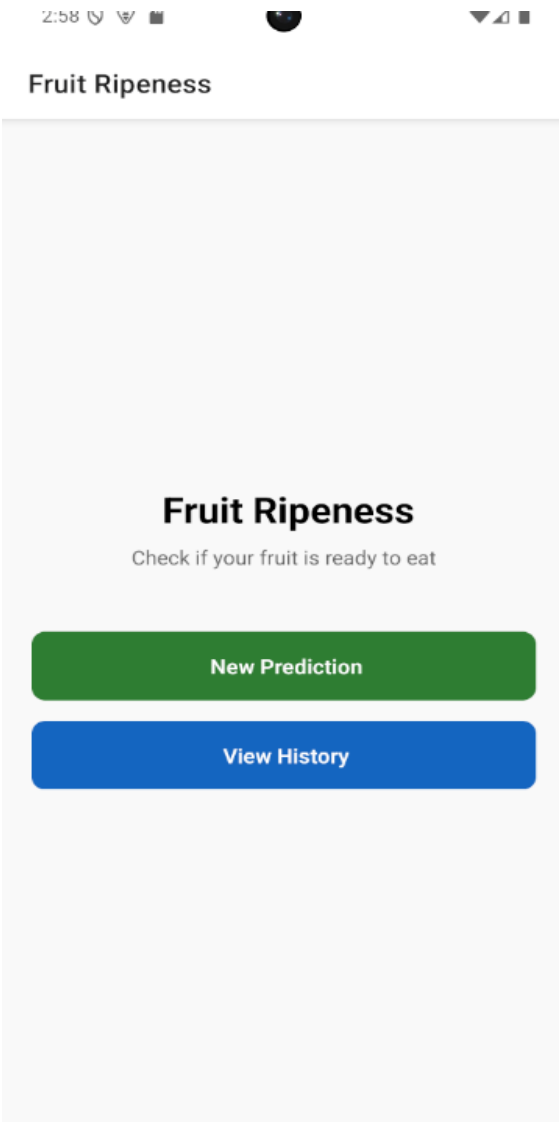
Year 4

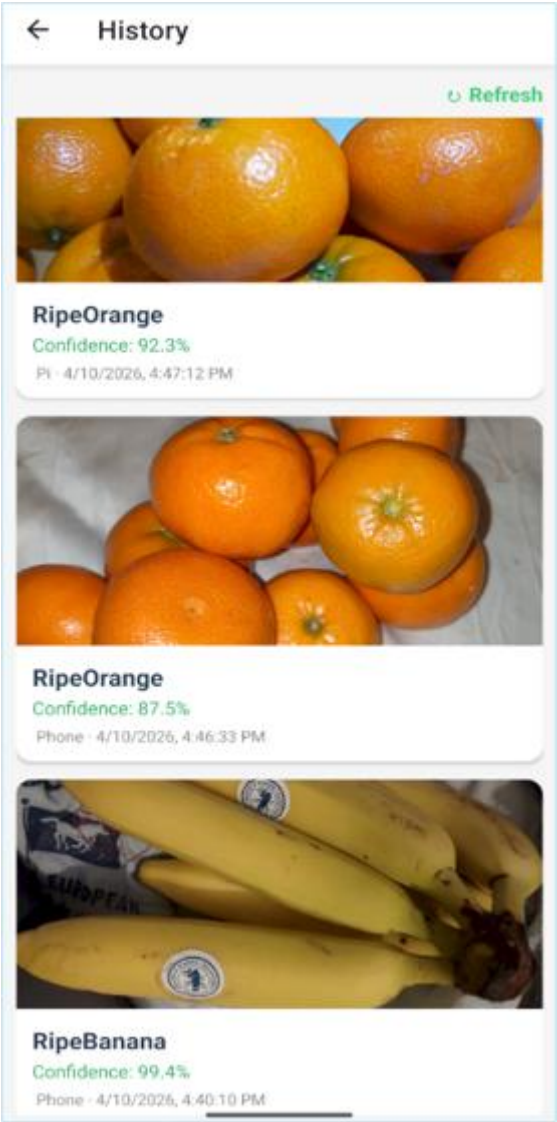
Rory O Loughlin

Bachelor of Engineering (Honours) in Software and Electronic Engineering

Atlantic Technological University Galway

2025/2026


2:58
Fruit Ripeness
Fruit Ripeness
Check if your fruit is ready to eat
New Prediction
View History


History
Refresh
RipeOrange
Confidence: 92.3%
Pi · 4/10/2026, 4:47:12 PM
RipeOrange
Confidence: 87.5%
Phone · 4/10/2026, 4:46:33 PM
RipeBanana
Confidence: 99.4%
Phone · 4/10/2026, 4:40:10 PM

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

_____

Acknowledgements

I want to thank my project supervisor, Michelle Lynch for her assistance throughout the academic year.

Table Of Contents

Table Of Contents 5
1 Summary 6
2 Poster 7
3 Introduction 8
4 Technologies used 9
5 Project Architecture 12
6 Project management 13
7 Project Code 17
8 Testing 38
9 Problems faced and Solutions 40
10 Ethics 42
11 AI Use 43
12 Conclusion 44
13 References 45

1 Summary

For my final year project, I wanted to make a computer vision-based project with a purpose and that could assist the user with their task.

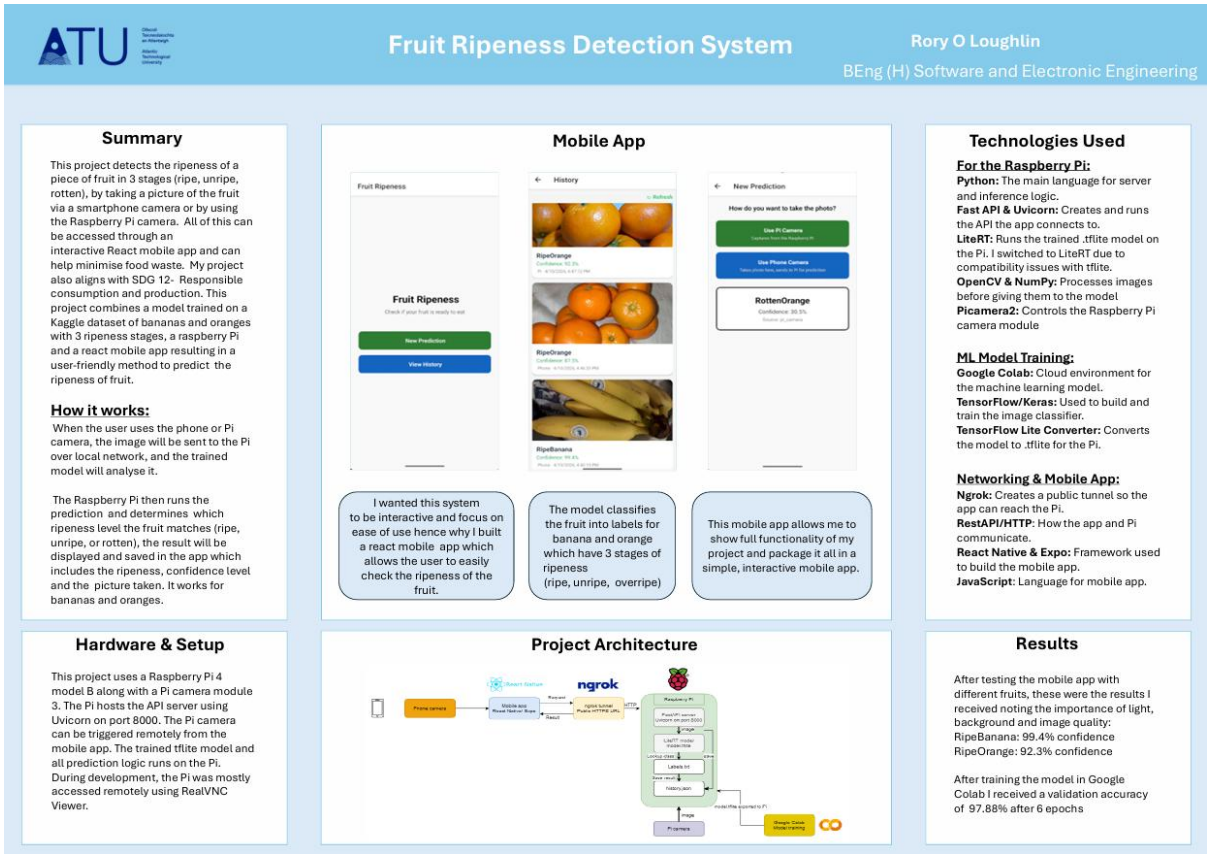
My fruit ripeness detection system allows the user to quickly and easily detect the ripeness of a piece of fruit via a user-friendly mobile app.

The user scans the QR code generated by Expo Go to open the mobile app on their smartphone. Once opened, they are presented with the home page and ease of use and simplicity is important as everyone should be able to navigate through the app with ease. The user has the option to select “New Prediction” or “View History”. If the user selects “New Prediction”, the user will be navigated to a new screen with two options, to take a picture using the smartphone camera, or to capture an image using the Raspberry Pi camera. When the image is captured, a prediction will be given along with a confidence level and the image captured will be displayed. The user can view previous predictions by pressing the “View History” button in the home screen. This makes for a seamless, user-friendly experience.

I used Jira to track and manage the project along with documenting the progress, I used OneNote to keep logs of my progress and logging my weekly standup. Key technologies I used in my project are Python which most of my project is written with, I used FastAPI and Uvicorn to create and run an API the app can connect to. I used LiteRT to run the trained model on the Raspberry Pi, OpenCV and NumPy for processing images, and Picamera2 to control the Pi camera module. Technologies used for model training are Google Colab where I trained the model, TensorFlow and Keras to build and train the image classifier, I used TensorFlow Lite Converter to convert the model to .tflite for the Pi. For the mobile app I use ngrok to create an HTTPS tunnel that allows the mobile app to communicate with the Pi over a REST API. React Native and Expo are used to build the mobile app, and I built the mobile app using JavaScript.

Through this project I have improved my problem-solving skills by advancing through the project and my project management skills have also improved from using Jira and OneNote. I have explored multiple different areas to complete this project including training a dataset, building a mobile app, understanding the Raspberry Pi and understanding how all these can be linked together successfully. This challenged me and forced me to become a better engineer.

2 Poster


Fruit Ripeness Detection System
Rory O Loughlin
BEng (H) Software and Electronic Engineering
Summary
This project detects the ripeness of a piece of fruit in 3 stages (ripe, unripe, rotten), by taking a picture of the fruit via a smartphone camera or by using the Raspberry Pi camera. All of this can be accessed through an interactive React mobile app and can help minimise food waste. My project also aligns with SDG 12- Responsible consumption and production. This project combines a model trained on a Kaggle dataset of bananas and oranges with 3 ripeness stages, a raspberry Pi and a react mobile app resulting in a user-friendly method to predict the ripeness of fruit.
How it works:
When the user uses the phone or Pi camera, the image will be sent to the Pi over local network, and the trained model will analyse it.
The Raspberry Pi then runs the prediction and determines which ripeness level the fruit matches (ripe, unripe, or rotten), the result will be displayed and saved in the app which includes the ripeness, confidence level and the picture taken. It works for bananas and oranges.
Mobile App
Fruit Ripeness
New Prediction
View History
History
RipeOrange
RipeOrange
RipeBanana
New Prediction
How do you want to take the photo?
Use Pi Camera
Use Phone Camera
RottenOrange
I wanted this system to be interactive and focus on ease of use hence why I built a react mobile app which allows the user to easily check the ripeness of the fruit.
The model classifies the fruit into labels for banana and orange which have 3 stages of ripeness (ripe, unripe, overripe)
This mobile app allows me to show full functionality of my project and package it all in a simple, interactive mobile app.
Technologies Used
For the Raspberry Pi:
Python: The main language for server and inference logic.
Fast API & Uvicorn: Creates and runs the API the app connects to.
LiteRT: Runs the trained .tflite model on the Pi. I switched to LiteRT due to compatibility issues with tflite.
OpenCV & NumPy: Processes images before giving them to the model
Picamera2: Controls the Raspberry Pi camera module
ML Model Training:
Google Colab: Cloud environment for the machine learning model.
TensorFlow/Keras: Used to build and train the image classifier.
TensorFlow Lite Converter: Converts the model to .tflite for the Pi.
Networking & Mobile App:
Ngrok: Creates a public tunnel so the app can reach the Pi.
RestAPI/HTTP: How the app and Pi communicate.
React Native & Expo: Framework used to build the mobile app.
JavaScript: Language for mobile app.
Hardware & Setup
This project uses a Raspberry Pi 4 model B along with a Pi camera module 3. The Pi hosts the API server using Uvicorn on port 8000. The Pi camera can be triggered remotely from the mobile app. The trained tflite model and all prediction logic runs on the Pi. During development, the Pi was mostly accessed remotely using RealVNC Viewer.
Project Architecture
ngrok
Results
After testing the mobile app with different fruits, these were the results I received noting the importance of light, background and image quality:
RipeBanana: 99.4% confidence
RipeOrange: 92.3% confidence
After training the model in Google Colab I received a validation accuracy of 97.88% after 6 epochs

3 Introduction

After researching project ideas, the idea of creating a computer vision-based project caught my eye because there are many possibilities to choose from. I personally thought a computer vision-based project which involves the detection of the ripeness level of a piece of fruit would make for a good project and it could assist physically impaired individuals with determining the ripeness of a piece of fruit such as a person with a form of blindness such as colour blindness. It could also reduce the amount of food being wasted.

To make the project as user friendly as possible, building it around a mobile app instead of a web app made more sense to me because in a real-world scenario the user will be taking pictures from their smartphone and be accustomed to doing so, therefore, having the ability to capture an image via smartphone is important.

This would prove to be a learning curve for me because I have not explored many of these technologies I was planning to use yet such as the mobile app or setting up the Raspberry Pi. This project would expand my knowledge in areas I had not explored.

Below are the requirements made to define what the project needed to achieve.

- The system needs to be able to classify a piece of fruit as ripe, unripe or overripe. These 3 stages would cover the three most important stages of ripeness for the fruit.
- The system must be able to capture an image from the raspberry Pi camera and the phone camera. This gives the user two options for how to get a prediction. The Pi camera is better suited for a fixed setup whereas the phone is more versatile, and a good picture can be taken anywhere which helps for real world scenarios.
- The system must store previous predictions to allow the user to view them and to see if the prediction makes sense for the image captured and the user can track the fruit ripeness over time.
- The mobile app must be user-friendly so users with a lack of experience with technology can easily navigate through the app.
- The API must return a result in under 10 seconds to ensure acceptable levels of responsiveness from the application.

- The model should achieve 80% validation accuracy to ensure the predictions returned are consistently accurate.

4 Technologies used

To transform this idea into a functioning project I had to gain an understanding of the technologies I would need to implement to complete the project.

4.1 Python

Python is a popular programming language with many different use cases, and it is simple and readable. [1] I used Python to write the FastAPI server in the Raspberry Pi's OS which allows the Pi to use the trained dataset to make a prediction, I trained an image dataset using Google Colab which was all written through Python.

4.2 Uvicorn

Uvicorn [2] is a lightweight web server for Python, and I use it in this project to run the FastAPI server on the Raspberry Pi on port 8000. Uvicorn allows my server to stay alive while the Pi is running, therefore allowing the Pi to receive requests from the mobile app.

4.3 FastAPI

FastAPI [3] is a Python web framework that allows the user to construct APIs. It automatically produces documentation for all endpoints improving the useability of this technology, this framework is popular for building REST APIs in Python. I use it to create an API the mobile app can communicate with. It exposes the endpoints I created in the server on the Pi.

4.4 OpenCV

OpenCV [4] is a computer vision library in Python. I would need this to configure the Pi's camera, and I use it for decoding image bytes, converting images from BGR to RGB colour format and to resize images to 224x224 so they match the input size the trained model expects.

4.5 NumPy

NumPy [5] is a Python library for computing numbers and has the ability to handle large multi-dimensional arrays and matrices, it also can compute many mathematical

functions, I have implemented this to allow me to convert image data into a numerical array that can be used in the model and I use it to find the highest confidence score in the models output when predicting the class.

4.6 Picamera2

Picamera2 [6] is a library for controlling the raspberry Pi camera and I use this library for making configurations such as configuring the camera for a 1280x720 photo, enabling continuous autofocus and saving the image to a temporary file before sending it to the model to get a prediction.

4.7 Google Colab

Google Colab [7] is a cloud-based Python environment which allows the user to create notebooks and grants access to cloud computing resources such as GPUs which makes it an ideal candidate for training an image dataset. I used this to train the model, and it sped up the training times due to the access to a GPU which was more efficient than training locally.

4.8 TensorFlow and Keras

TensorFlow [8] is an open-source machine learning framework made by Google. Keras [9] is an API built on top of TensorFlow that makes it easier to train neural networks. In the case of my project, I used these to build and train the fruit ripeness classification model. This method allowed me to train the model quicker because the MobileNetV2 model I loaded from Keras was pretrained, I then added a custom classification head on top for quicker training times compared to training a large model from scratch.

4.9 MobileNetV2

MobileNetV2 [10] is a neural network architecture made by Google, it uses a technique called transfer learning. This is when a model that has been pretrained on a large dataset can be changed to suit a new classification task that has less data and training time. I used MobileNetV2 as the base model for the fruit ripeness classification. This method allowed me to avoid an overly large dataset that would take too much time to train.

4.10 TensorFlow Lite Converter

The TensorFlow Lite converter is a tool included in TensorFlow that converts a trained Keras model into the .tflite format. I personally implemented this converter after the training to export the model as model.tflite which allowed it to be transferred to the Raspberry Pi smoothly where it is later loaded and run by LiteRT.

4.11 React Native

I used React Native [11] to construct the Mobile app along with the knowledge and experience I gained from the Mobile App Development module, and I was able to trigger the predictions, view the results and browse through the prediction history, all through a mobile app.

4.12 Expo

Expo [12] is a set of tools built around React Native that simplifies the development and testing of mobile apps. Apps can be tested on a device without needing to build a native binary. I used Expo to access an Android emulator which allowed me to test the mobile app quickly and easily in real time without having to connect a smartphone to the app.

4.13 Ngrok

Ngrok [13] is a tool that creates a secure public HTTPS tunnel to a locally running server. This allows a server running on a local machine to be accessed from anywhere on the internet and it avoids configuring port forwarding or a static IP address. I am using ngrok in this project to create a reliable bridge between the Raspberry Pi and the React mobile app so they can communicate. Ngrok allows me to connect the Raspberry Pi to the React mobile app even if there are network changes.

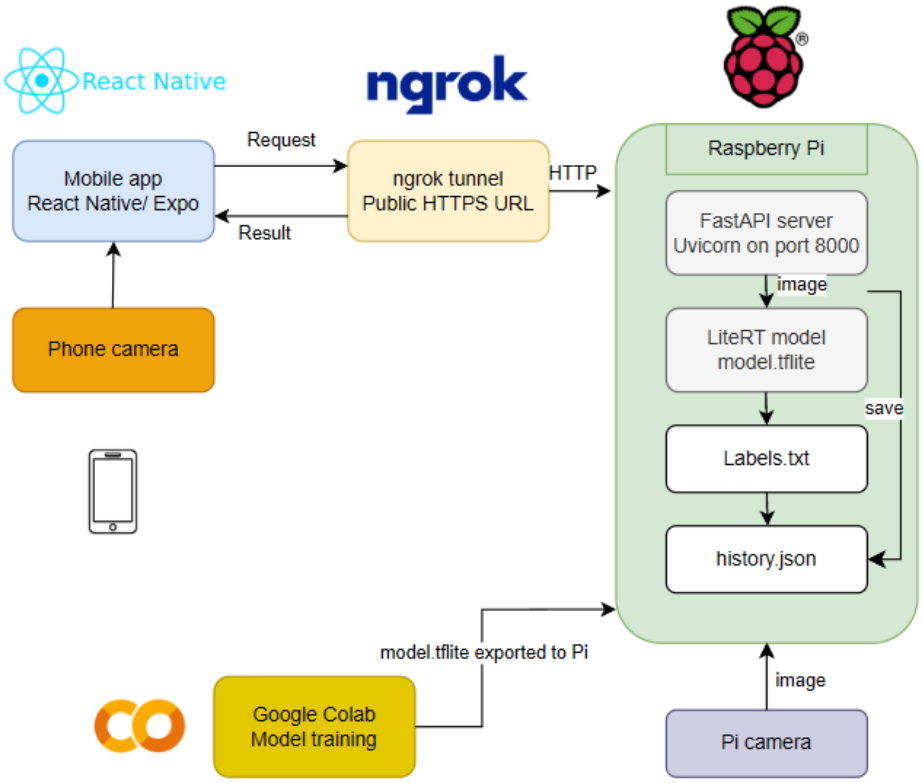
4.14 RealVNC Viewer

Real VNC Viewer [15] is a remote desktop application which allows the user to view and control another computer's desktop over a network connection. I use RealVNC viewer to gain access to the Raspberry Pi desktop remotely from a Windows PC. This allowed me access to the Pi's OS without having to connect peripherals to the Pi and needing extra hardware.

4.15 LiteRT

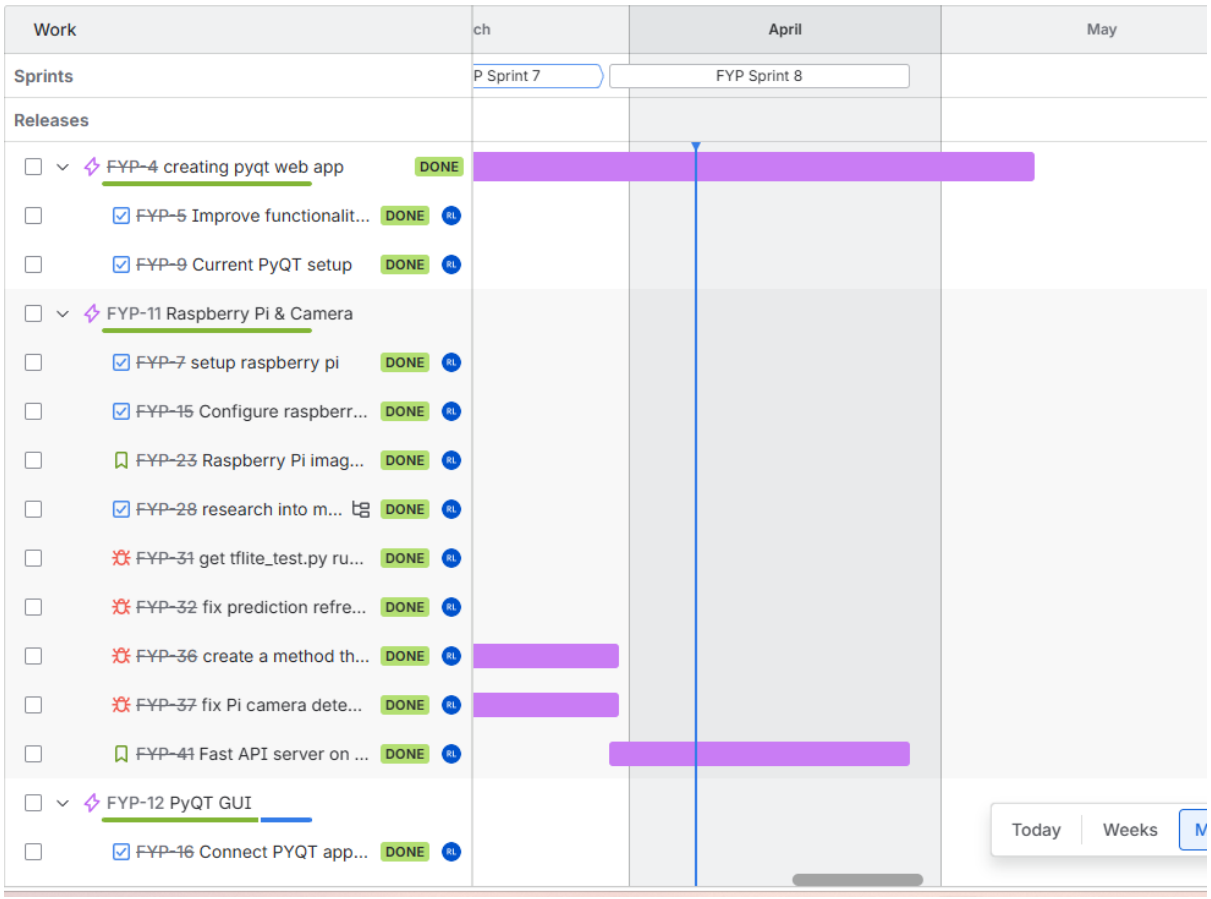
LiteRT [14] is a runtime for running machine learning models on edge devices. I use LiteRT through the ai_edge_litert package on the Raspberry Pi to load and run the trained model.

5 Project Architecture



6 Project management

I used Jira [16] to manage and document my project progress throughout the academic year and regularly updated it with newly completed work, new work to complete, bugs in need of fixing. I created epics which I could link these tasks to which epic was most fitting for the task. I would set each sprint to be 4 weeks long.



I used OneNote to document my weekly progress on tasks in a more informal way, and my weekly standups were documented here also. I had a weekly AI prompt log to show and keep track of AI prompts used for the project.

Add Page

Week 1
Week 2
Week 3
Week 4
Week 5
Week 6
Reading week 7
Week 8
Week 9
Week 10
Week 11
Week 11
Week 12
Sem 2 week 1
Sem 2 Week 2
Sem 2 week 3
Sem2 week 4
Sem2 Week 5
Sem 2 Week 6
Sem 2 week 7
Sem 2 week 8
Sem2 week 9
Sem2 week 10
Easter Break
Sem 2 week 11
Sem 2 week 12

Easter Break

31 March 2026 12:43

Plans:
Finnish the poster
Finnish the report
Ensure all work is correctly documented, no bugs, fully functional and pushed to GitHub
Pushed the latest iteration of the fastAPI server onto GitHub:
I have created a FastAPI server on the Raspberry PI so the local mobile app can send a request to the Pi and the prediction will run on the Pi's OS. I added the labels.txt and model.tflite into the filesystem in the Pi's OS and the server reads the labels.txt and loads the class names (ripe, unripe, rotten) and it loads the trained model into memory using LiteRT, it reads the models expected input size and data type from the model file.
There are four main endpoints in the server:
Status: This is a health check to see if the Pi can be reached
Capture: Uses the Pi camera to take a picture, then run the prediction and save the result to history and return the prediction and confidence
Predict: Uses a photo taken from a smartphone camera when the user chooses to take a picture using mobile phone and runs the prediction logic and saves the prediction along with image captured and prediction and confidence level
History: Returns the alst 20 predictions stored in history.json so the user can see the previous 20 predictions, this also aids debugging and testing.
I was able to structure this by researching into the FastAPI documentation and watching informative YouTube videos
references:
Python Software Foundation (2024). *Python 3 standard library documentation*. https://docs.python.org/3/library/
van Rossum, G., Warsaw, B. & Coghlan, A. (2001). *PEP 8 — Style guide for Python code*. Python Software Foundation. https://peps.python.org/pep-0008/
OpenCV team (2024). *OpenCV 4.x documentation — Image processing*. https://docs.opencv.org/4.x/
Ramírez, S. (2024). *FastAPI documentation*. Tiangolo. https://fastapi.tiangolo.com
Harris, C.R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. https://doi.org/10.1038/s41586-020-2649-2
https://www.tensorflow.org/lite/examples/object_detection/overview
Raspberry Pi Ltd (2023). *The Picamera2 Library*. https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf
https://github.com/raspberrypi/picamera2/tree/main/examples

I also debugged the image prediction inaccuracy issues below:

I also removed a few lines of code in the api_server.py in the Pi's OS to avoid double normalisation, which I thought could have been causing the inaccuracy issues so I got rid of the if statement below where the batch dimensions are

I pushed all my work and changes made to GitHub so all my updated work would be in one place.

Final-Year-Project Public

Pin | Watch 0

main | 3 Branches | 0 Tags | Go to file T | Add file | Code

rory619 Refactor POST request in integration test — 319d8ae · 19 hours ago — 51 Commits

FruitRipenessApp	added image picker button that allows capturing an image o...	last month
app	FYP-25: Add Tkinter GUI capture + predict script	2 months ago
assets/sample_images	Committing the initial project	6 months ago
ui	FYP-25: Add Tkinter GUI capture + predict script	2 months ago
.gitignore	Committing the initial project	6 months ago
FruitRipenessV2.ipynb	FYP-25: Add Tkinter GUI capture + predict script	2 months ago
How I setup the pi OS	explaining How I setup the Pi	5 months ago
api_server.py	Adding Latest version of the FastAPI server on the Pi to GitHub	last month
jira=httpshome.atlassian.comojj9d73.txt	FYP-25: Add Tkinter GUI capture + predict script	2 months ago
requirements.txt	Added camera feature that enables the user to open camera ...	5 months ago
test_api.py	Refactor POST request in integration test	19 hours ago

Add Page

Untitled page
Sem 2 week1
Sem 2 week 2
Sem 2 week3
Sem 2 week4
Sem 2 week 5
Sem 2 week 6
Sem 2 week 7
Sem 2 week 8
Sem 2 week 9
Sem 2 week 10
Easter Break
Sem 2 week 11
Sem 2 week 12

Sem 2 week 6

Monday 23 February 2026 12:22

what are the differences between running a python inference on vscode locally and running it on the raspberRy pi's OS
Here's what *actually* changes when you run the **same Python inference code** in VS Code on your laptop/desktop vs on the **Raspberry Pi's OS.**

1) Hardware + performance envelope

VS Code locally (PC/laptop)

- Typically **much faster CPU** (higher clocks, more cores, better cache).
- Often has **more RAM** and faster storage.
- May have **GPU/NPU options** (even if you're not using them yet).
- Result: lower latency per prediction, easier to run larger models.

Raspberry Pi OS (Pi Zero 2 W / Pi 4 / etc.)

- **ARM CPU**, usually lower clock and fewer resources (Zero 2 W is especially tight).
- Less RAM, slower storage (microSD), more sensitive to swapping/IO.
- Result: inference is usually slower unless the model is **quantized** and lightweight.

2) CPU architecture + package availability (x86_64 vs ARM)

PC: x86_64 wheels are common → installing TensorFlow/PyTorch/OpenCV is usually easier.
Pi: ARM wheels can be limited → you often use:

- tflite-runtime / LiteRT instead of full TensorFlow,
- Pi-friendly OpenCV builds,
- careful version pinning.

This impacts **setup effort** and sometimes which optimizations you get "for free".

3) Runtime differences (TensorFlow vs TFLite Runtime / LiteRT)

On a PC you might run:

- tensorflow (full) or tflite inside TensorFlow.

On the Pi you typically run:

- **LiteRT / tflite-runtime** style interpreter (smaller footprint, faster install, fewer dependencies).

Practical impact:

- Your model format is the same (.tflite), but **imports, delegates, and threading defaults** can differ.
- Performance tuning (threads, XNNPACK availability) matters more on Pi.

4) Input pipeline differences (camera + image handling)

PC local inference

- Often uses a **webcam (OpenCV)** or file uploads.
- Latency includes USB camera + decode + resize.

Pi inference

- Can use **Picamera2/libcamera** directly (more "native" camera path).
- Often faster/cleaner capture pipeline on Pi hardware *if configured well*.
- Also easier to run headless (no GUI) and expose /predict over LAN.

5) Deployment model: "app on your machine" vs "device service"

VS Code local

- You run scripts interactively.
- Great for debugging, iteration, visualizing images, quick experiments.

Pi OS

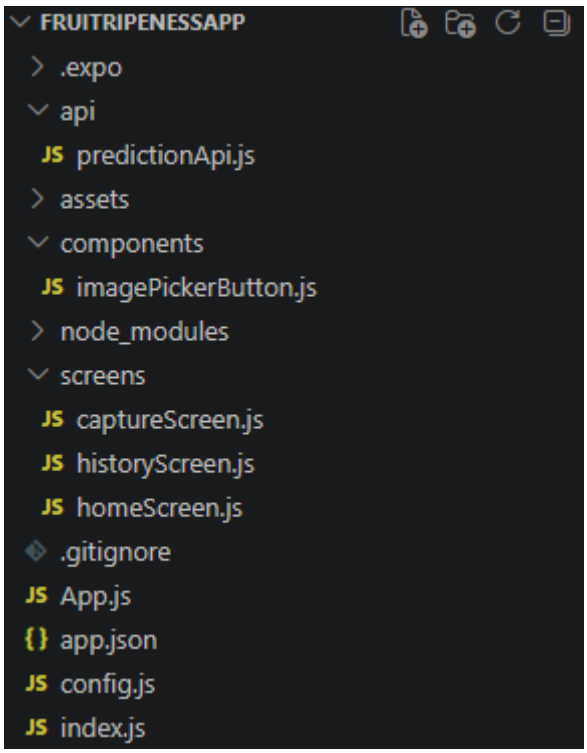
- You're usually building a **service:**
  - always-on inference endpoint (FastAPI),

7 Project Code

I will break down all the code made in this project, this includes the Colab notebook where I trained the image dataset, the code used to build the React mobile app and the FastAPI server on the Raspberry Pi.

7.1 Front end

The front end contains all the code for the React Native mobile app. I have it structured into 3 folders, screens which contains the three app screens, api which contains all the network calls to the Pi and components, which creates a camera button component. The entry point is through index.js, the navigation structure is created in app.js, and the ngrok connection is set in config.js.



Index.js acts as the Expo entry point for the app allowing the user to view the app via Expo Go.

```js
// index
// - Purpose: Expo entry point. Registers the root component for the app runtime environment.

import { registerRootComponent } from 'expo';

import App from './App';

// registerRootComponent calls AppRegistry.registerComponent('main', () => App);
// It also ensures that whether you load the app in Expo Go or in a native build,
// the environment is set up appropriately
registerRootComponent(App);

```

Config.js defines the ngrok URL for the Raspberry Pi server which allows the Raspberry Pi and the mobile app to communicate.

```js
export const PI_URL = 'https://crazedly-bloomiest-melonie.ngrok-free.dev';
```

The navigation structure is created in **App.js**, using createNativeStackNavigator to create three screens, Home, Capture and History. When the user enters the app, the user will enter the home screen by default because initialRouteName is set to Home. React navigation will choose the correct component to render because each Stack.Screen maps a screen name to its component.

```js
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

import HomeScreen    from './screens/homeScreen';
import CaptureScreen from './screens/captureScreen';
import HistoryScreen from './screens/historyScreen';

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen name="Home"    component={HomeScreen}    options={{ title: ' Fruit Ripeness' }} />
        <Stack.Screen name="Capture" component={CaptureScreen} options={{ title: 'New Prediction' }} />
        <Stack.Screen name="History" component={HistoryScreen} options={{ title: 'History' }} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

The app is designed to be user friendly so anyone can navigate through the app with ease. The Home screen displays 2 buttons to choose from; one brings the user to the Capture screen while the other brings the user to the History screen. Each button is a Touchable Opacity which allows for elements to be pressed like a button in the app. When these buttons are pressed the name of the destination screen is called by calling navigation.navigate().

```
import React from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';

export default function HomeScreen({ navigation }) {
  return (
    <View style={styles.screen}>

      <Text style={styles.title}> Fruit Ripeness</Text>
      <Text style={styles.subtitle}>Check if your fruit is ready to eat</Text>

      <TouchableOpacity
        style={styles.greenButton}
        onPress={() => navigation.navigate('Capture')}
      >
        <Text style={styles.buttonText}> New Prediction</Text>
      </TouchableOpacity>

      <TouchableOpacity
        style={styles.blueButton}
        onPress={() => navigation.navigate('History')}
      >
        <Text style={styles.buttonText}> View History</Text>
      </TouchableOpacity>

    </View>
  );
}
```

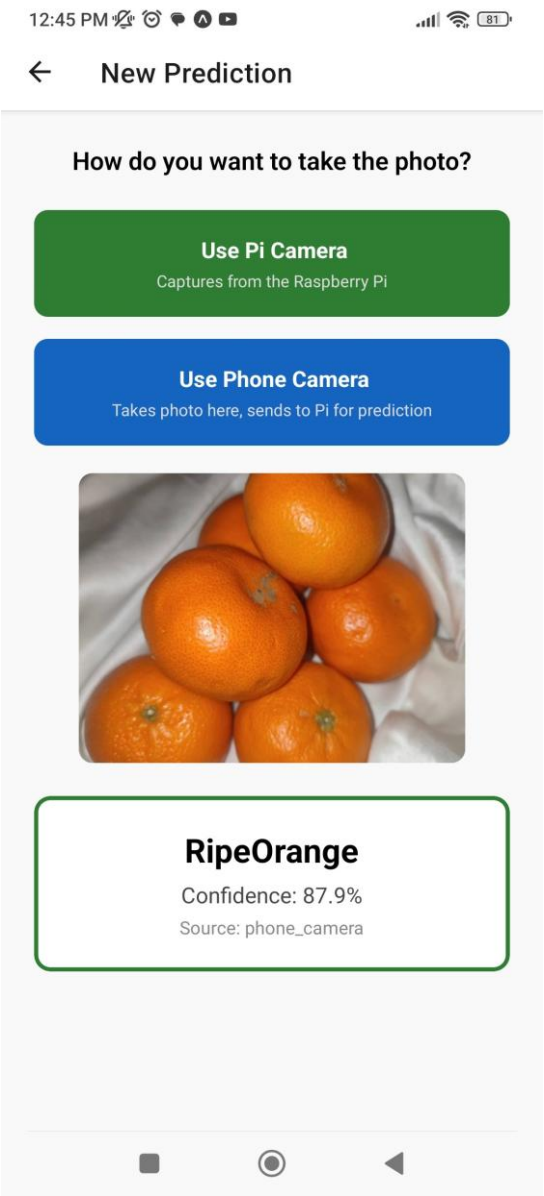
CaptureScreen.js displays 2 methods to get a fruit ripeness prediction. The first option is by using the Pi camera. When this button is pressed, the usePiCamera function runs which calls the image capture function from the FastAPI server running on the Pi's OS. This function then sends a GET request to the /capture endpoint on the Pi which makes the Pi take a picture using Picamera2. From there the server running on the Pi will run the prediction and return the result. The result is stored in the result state variable and displayed in the results section.

```
import React, { useState } from 'react';
import {
  View, Text, TouchableOpacity, Image,
  ActivityIndicator, StyleSheet, Alert, ScrollView
} from 'react-native';
import * as ImagePicker from 'expo-image-picker';
import { captureWithPiCamera, predictWithPhonePhoto } from '../api/predictionApi';

export default function CaptureScreen() {
  const [photo, setPhoto]     = useState(null);   // URI of the phone
  const [result, setResult]   = useState(null);   // { label, score, source }
  const [loading, setLoading] = useState(false);

  // --- Option 1: Pi camera does everything ---
  async function usePiCamera() {
    setLoading(true);
    setResult(null);
    setPhoto(null);
    try {
      const data = await captureWithPiCamera();
      setResult(data);
    } catch (err) {
      Alert.alert('Error', `Could not reach the Pi: ${err.message}`);
    } finally {
      setLoading(false);
    }
  }
```

When the user chooses to take a picture using the phone camera this will run a separate function called usePhoneCamera which will activate the camera on the phone. It requests permission to access the phone's camera first then the camera is opened using launchCameraAsync. Once the user takes a picture, the image URL is stored in the photo state variable, and a preview is shown in the app. From there, the URL is passed to predictionAPI.js which sends the image to the Pi's prediction endpoint.

```
// --- Option 2: Phone takes photo, Pi predicts ---
async function usePhoneCamera() {
  // Ask for camera permission
  const { status } = await ImagePicker.requestCameraPermissionsAsync();
  if (status !== 'granted') {
    Alert.alert('Permission needed', 'Please allow camera access.');
    return;
  }

  // Open the camera
  const picked = await ImagePicker.launchCameraAsync({
    allowsEditing: true,
    aspect: [4, 3],
    quality: 0.7,
  });

  if (picked.canceled) return;

  const uri = picked.assets[0].uri;
  setPhoto(uri);
  setResult(null);
  setLoading(true);

  try {
    const data = await predictWithPhonePhoto(uri);
    setResult(data);
  } catch (err) {
    Alert.alert('Error', `Prediction failed: ${err.message}`);
  } finally {
    setLoading(false);
  }
}
```

Once a result comes back, it will be displayed, showing the label, confidence score as a percentage and the image, the border colour will change based on the stage of ripeness, green for ripe, orange for unripe and red for overripe.



The **History screen** will show the last 20 predictions stored on the Pi by calling the history endpoint made on the Pi which stores the returned array in the history state variable. The list is rendered in the app by using a FlatList.

```
import React, { useState, useCallback } from 'react';
import {
  View, Text, FlatList, StyleSheet,
  TouchableOpacity, Image, ActivityIndicator
} from 'react-native';
import { useFocusEffect } from '@react-navigation/native';
import { getHistory } from '../api/predictionApi';

export default function HistoryScreen() {
  const [history, setHistory] = useState([]);
  const [loading, setLoading] = useState(false);

  // Fetch the list of predictions from the Pi
  async function fetchHistory() {
    setLoading(true);
    try {
      const data = await getHistory();
      setHistory(data);
    } catch (err) {
      console.error('Could not load history:', err);
    } finally {
      setLoading(false);
    }
  }
```

The list will update every time the user navigates to this screen by implementing useCallback to re-run fetchHistory each time the screen is reloaded.

```
// Re-fetch every time the user navigates to this screen
useFocusEffect(useCallback(() => { fetchHistory(); }, []));

// Renders a single prediction
function renderCard({ item }) {
  const time   = new Date(item.timestamp).toLocaleString();
  const source = item.source === 'phone_camera' ? ' Phone' : ' Pi';

  return (
    <View style={styles.card}>


      {item.image ? (
        <Image
          source={{ uri: `${item.image}` }}
          style={styles.photo}
          resizeMode="cover"
        />
      ) : (
        <View style={styles.noPhoto}>
          <Text style={styles.noPhotoText}>No image</Text>
        </View>
      )}


      <View style={styles.cardBody}>
        <Text style={styles.label}>{item.label}</Text>
        <Text style={styles.score}>Confidence: {(item.score * 100).toFixed(1)}%</Text>
        <Text style={styles.meta}>{source} · {time}</Text>
      </View>

    </View>
  );
}

return (
  <View style={styles.screen}>
```

The images attached to these predictions in the list are shown by using the base64 string embedded in the JSON and a grey placeholder is shown if there is no image attached.

I kept all the HTTP communication in one file which is **predictionAPI.js**. There are three functions in this file which get exported which are captureWithPiCamera, which allows image capture, PredictWithPhonePhoto which creates a data object with the image which is used for the prediction and getHistory reaches the history endpoint in the server running on the Pi. All these functions check if the response status is ok and have error handling if the status is not ok.

```
import { PI_URL } from '../config';

// Tell the Pi camera to capture a photo and predict ripeness.
// Returns:  label, score, source, timestamp
export async function captureWithPiCamera() {
  const res = await fetch(`${PI_URL}/capture`);
  if (!res.ok) throw new Error(`Pi returned status ${res.status}`);
  return res.json();
}

// Send a photo taken on the phone to the Pi for prediction.
// Returns:  label, score, source, timestamp
export async function predictWithPhonePhoto(uri) {
  const form = new FormData();
  form.append('file', {
    uri,
    name: 'photo.jpg',
    type: 'image/jpeg',
  });

  const res = await fetch(`${PI_URL}/predict`, {
    method: 'POST',
    body: form,
  });
  if (!res.ok) throw new Error(`Pi returned status ${res.status}`);
  return res.json();
}

// Fetch all past predictions stored on the Pi.
// Returns: array of  label, score, source, timestamp, image
export async function getHistory() {
  const res = await fetch(`${PI_URL}/history`);
  if (!res.ok) throw new Error(`Pi returned status ${res.status}`);
  return res.json();
}
```

I created ImagePickerButton.js so I could encapsulate the camera button and thumbnail preview. The code accepts three props, Image uses the current picture URL, or it returns null, onChange callback is used along with the new URL after the picture is taken and disabled would be set to true while a request is happening. Once the camera permission is granted, onChange is called with the result. After the picture is taken, a thumbnail is added next to the button.

7.2 Backend

For the backend I created a Python file that runs as a FastAPI server on the Raspberry Pi's OS which communicates with the front end via ngrok and it is started using Uvicorn on port 8000. This server runs everything on the Pi such as getting requests from the mobile app, taking pictures using the Pi camera, running the ripeness prediction using the trained model, saving the results to a history file and sending the results back to the app. I exported the trained model along with the labels.txt and added them into the file system on the Pi by using SCP which allowed me to run the image prediction process on the Pi.

All the important file paths are defined at the top of the code using pathlib.Path and the capture directory is created automatically upon startup using mkdir(exist_ok=True) so it will be created if it does not already exist and it will throw an error if it does exist.

```
# File paths

BASE_DIR     = Path("/home/rory/fruitchecker/inference")
CAPTURES_DIR = BASE_DIR / "captures"
MODEL_PATH   = BASE_DIR / "model.tflite"
LABELS_PATH  = BASE_DIR / "labels.txt"
HISTORY_FILE = BASE_DIR / "history.json"

# Create the captures folder
CAPTURES_DIR.mkdir(exist_ok=True)


# Load class labels from labels.txt (one label for every line)

labels = [
    line.strip()
    for line in LABELS_PATH.read_text().splitlines()
    if line.strip()
]
```

The class labels are loaded from labels.txt from startup by reading the contents of the file. This gives a Python list where each label matches the corresponding output score from the model. The TFLite model is loaded using LiteRT's interpreter class. The input and output tensor details are read from the model after allocate_tensors is called. I initially tried to use tflite-runtime, but I ran into compatibility issues when trying to run the server which led me to switch to ai_edge_litert.

```
# Load the TFLite model using LiteRT

interpreter = Interpreter(model_path=str(MODEL_PATH))
interpreter.allocate_tensors()
```

“run_inference(image_bytes)” is the function that gets called from the mobile app to make the Pi give a prediction. The function changes raw image bytes into a NumPy array. np.frombuffer is used to turn the bytes into a NumPy buffer that OpenCV can use.

OpenCV then loads images in BGR colour by default, but the model was trained on RGB images. cv2.cvtColor is then used to convert from BGR to RGB before the processing happens.

cv2.resize scales the image to match the exact dimensions the model expects which were read from the model’s input tensor details at the startup.

“np.expand_dims” adds another dimension at the front, therefore, the image shape becomes (1, height, width, 3). I needed this because the model expected a batch of images.

From there, the prepared image is then loaded into the models input slot using “set_tensor”, then “invoke” runs the model and “get_tensor” will read the output scores which will be one score per class. After that, “np.argmax” is used to get the highest score.

```
def run_inference(image_bytes: bytes) -> dict:
    # Decode the image bytes into an OpenCV image array

    frame = cv2.imdecode(np.frombuffer(image_bytes, np.uint8), cv2.IMREAD_COLOR)

    # Convert from BGR (OpenCV default) to RGB (what the model expects)
    rgb_image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    # Resize the image to match the model's expected input size
    rgb_image = cv2.resize(rgb_image, (input_width, input_height))

    # Add a batch dimension so the shape is now (1, height, width, 3)
    tensor = np.expand_dims(rgb_image, axis=0).astype(input_dtype)

    # Normalise pixel values to 0.0-1.0
    if input_dtype == np.float32:
        tensor = tensor / 255.0

    # Plug the image into the model and run prediction
    interpreter.set_tensor(input_details["index"], tensor)
    interpreter.invoke()

    # Read the output scores (one score per class)
    scores = interpreter.get_tensor(output_details["index"])[0]

    # Find the index of the highest score
    best_index = int(np.argmax(scores))
    best_score = float(scores[best_index])

    # Look up the label name, or fall back to a generic class name
    label = labels[best_index] if 0 <= best_index < len(labels) else f"class_{best_index}"

    return {"label": label, "score": round(best_score, 4)}
```

The next function “save_to_history(entry, image_bytes)” saves a prediction to history.json. The mobile app can show the image later without the need of a separate image endpoint because it reads the base64 string from the history and renders it from there.

```python
def save_to_history(entry: dict, image_bytes: bytes = None):
    # Option to embed the image as a base64 string so the app can display it
    if image_bytes:
        entry["image"] = base64.b64encode(image_bytes).decode("utf-8")

    # Load existing history, or start with new one if the file doesn't exist
    history = []
    if HISTORY_FILE.exists():
        history = json.loads(HISTORY_FILE.read_text())

    # Add the new entry to the top of the list
    history.insert(0, entry)
    # Keep only the 20 most recent entries
    history = history[:20]


    # Write the updated history back to the file
    HISTORY_FILE.write_text(json.dumps(history, indent=2))
```

I created four separate endpoints for checking the server is online, taking a picture using the Pi camera and getting a prediction, taking a picture on the phone and getting a prediction and showing the previous predictions in a list.

Get /status is a simple health check endpoint to ensure the server is online and the mobile app could call this upon startup to see if the Pi is reachable.

```python
@app.get("/status")
def status():
    return {"status": "online", "model": "fruit_ripeness"}
```

The Get /capture endpoint is used to take a picture using the Pi camera and run a prediction. A temporary file is created using tempfile to give the camera a path to save the image to. Picamera2 is imported inside the function because the import will only fail when this endpoint is called instead of crashing the whole server on startup. The camera is setup for a 1280x720 still photo. Continuous autofocus mode is enabled by setting AFMode. I used “time.sleep(2)” to give the camera enough time to focus before the photo is taken and the image is then saved to a temporary file by using “capture_file”.

```python
def capture_and_predict():
    # Create a temporary file to save the image
    tmp = tempfile.NamedTemporaryFile(suffix=".jpg", delete=False)
    tmp.close()

    camera = None

    try:
        from picamera2 import Picamera2

        camera = Picamera2()

        # Set up the camera for a still photo at 720p
        config = camera.create_still_configuration(main={"size": (1280, 720)})
        camera.configure(config)
        camera.start()

       #af mode = 2: continuous auto focus
        camera.set_controls({"AfMode": 2})

        # Give the camera a moment to settle before capturing
        time.sleep(2)

        # Save the photo to the temporary file
        camera.capture_file(tmp.name)

    except Exception as error:
        #  return an error if there is failur
        os.unlink(tmp.name)
        return JSONResponse({"error": f"Camera failed: {str(error)}"}, status_code=500)
```

I use a “try/except/finally” block to ensure the camera is always stopped. If the camera fails, the temporary file is deleted, and a 500-error response is returned.

After the image has been captured, the image bytes are read from the temporary file, then it is deleted and the bytes are sent to “run_inference”, then the result is saved to the history, and the JSON result is given.

```python
finally:
    # stop and close the camera, even if an error occurred
    if camera:
        try:
            camera.stop()
            camera.close()
        except Exception:
            pass

# Read the saved image and delete the temporary file
image_bytes = Path(tmp.name).read_bytes()
os.unlink(tmp.name)

# Run the fruit ripeness prediction
result = run_inference(image_bytes)

result["source"]    = "pi_camera"
result["timestamp"] = datetime.datetime.now().isoformat()

save_to_history(result, image_bytes)

return JSONResponse(result)
```

The Post /predict endpoint is simpler than the other image capture/prediction endpoint due to the fact the image is being taken from the phone instead of the Pi camera. FastAPI’s “UploadFile” type handles the file and “await file.read” will read all the bytes.

The bytes are sent straight to “run_inference” and like the other endpoint, a timestamp would then be added, the result gets saved and the JSON is returned.

```
@app.post("/predict")
async def predict_from_upload(file: UploadFile = File(...)):
    # Read the uploaded image bytes
    image_bytes = await file.read()

    # Run the fruit ripeness prediction
    result = run_inference(image_bytes)
    result["source"]    = "phone_camera"
    result["timestamp"] = datetime.datetime.now().isoformat()

    save_to_history(result, image_bytes)

    return JSONResponse(result)
```

The last endpoint will show the previous 20 predictions by returning the contents of history.json and it will return an empty list if the file does not exist yet.

```
@app.get("/history")
def get_history():
    if not HISTORY_FILE.exists():
        return []

    return json.loads(HISTORY_FILE.read_text())
```

7.3 Training the model

I chose to build the code to train the model in Google Colab since I already had experience using this platform from the Python module from 3rd year. I was able to bring some of the logic from the labs we completed during the module such as a lab which involved neural network image recognition using FMNIST and I was able to build on this knowledge. Colab has an integrated GPU which aided the training process. I trained the notebook using a fruit image dataset that I got from Kaggle [17] which originally contained seven fruits each with 3 stages of ripeness, there is about 800 images of fruit for each stage of ripeness, after further testing I narrowed the fruits down to just oranges and bananas which allowed for faster training. I will break the notebook up into sections to explain each part properly.

The notebook begins by importing NumPy, matplotlib and TensorFlow which are needed for plotting graphs such as training results, numerical operations and building and training the model itself. I also mount my drive early on because the image dataset is stored there.

```
import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

print("TensorFlow version:", tf.__version__)

TensorFlow version: 2.19.0
```

```
from google.colab import drive
drive.mount('/content/drive', force_remount=True)



CANDIDATES = [
    "/content/drive/MyDrive/Colab Notebooks/fruit ripeness/train",
    "/content/drive/My Drive/Colab Notebooks/fruit ripeness/train",
]

DATA_DIR = next((p for p in CANDIDATES if os.path.isdir(p)), None)
if DATA_DIR is None:
    raise RuntimeError(
        "Train folder not available. Checked:\n" + "\n".join(CANDIDATES)
    )

print("Using dataset folder:", DATA_DIR)
print("Class folders:", os.listdir(DATA_DIR))
```

```
Mounted at /content/drive
Using dataset folder: /content/drive/MyDrive/Colab Notebooks/fruit ripeness/train
Class folders: ['UnripeBanana', 'UnripeOrange', 'RottenOrange', 'RottenBanana', 'RipeOrange', 'RipeBanana']
```

The dataset is loaded by reading the images from a folder where each name in the folder is considered a class label by using "tf.keras.utils.image_dataset_from_directory". An 80/20 validation split is used, and all the images are resized to 224x224 to match the input size the MobileNetV2 expects. It is then loaded in batches of 32.

```
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
SEED = 123

train_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="training",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
)
```

Both datasets are then optimised using autotune and "prefetch(autotune)" prepares the next batch in the background while the model is training the current one. These aspects make training faster.

```
class_names = train_ds.class_names
NUM_CLASSES = len(class_names)

print("Classes:", class_names)
print("Number of classes:", NUM_CLASSES)

AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.cache().prefetch(AUTOTUNE)
val_ds   = val_ds.cache().prefetch(AUTOTUNE)
```

```
Found 3074 files belonging to 6 classes.
Using 2460 files for training.
Found 3074 files belonging to 6 classes.
Using 614 files for validation.
Classes: ['RipeBanana', 'RipeOrange', 'RottenBanana', 'RottenOrange', 'UnripeBanana', 'UnripeOrange']
Number of classes: 6
```

When the model gets built, a small data augmentation pipeline is applied to the training images using Keras's "Sequential" API. This means random horizontal flips and small rotations happen of up to 5%. This artificially increases the difference of training images the model sees to help prevent overfitting in attempt to have the model perform well on images it has not been trained on.

```
# Data augmentation
data_augmentation = keras.Sequential(
    [
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(0.05),
    ],
    name="data_augmentation",
)
```

MobileNetV2 is then loaded while implementing "include_top=False" so the original classification head is removed. The weights are pretrained on ImageNet which is a large image dataset that has over a million images. These weights are frozen by setting "base_model.trainable = False" This ensures they don't change during training. The base model should already know how to detect general visual features like edges, textures and shapes, so only the new top layers need to be trained on the fruit dataset.

```
# using a pre trained MobileNetV2
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet",
)
base_model.trainable = False  # freeze to execute faster
```

After that, a "Rescaling" layer is added that normalises pixel values to –1.0 to 1.0 range that MobileNetV2 expects, then a "GlobalAveragePooling2D" layer that reduces the base models output to a single vector, a "Dropout" layer is then implemented with a rate of 0.2 to minimise overfitting, then a "Dense" output layer with a softmax activation

and one node per class. The output nodes are the length of the list of the class names which are loaded from the dataset. These are set dynamically by using "NUM_CLASSES"

```
inputs = keras.Input(shape=(224, 224, 3))

x = data_augmentation(inputs)


x = layers.Rescaling(1./127.5, offset=-1.0)(x)

x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.2)(x)
outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

model = keras.Model(inputs, outputs)
```

The model is then compiled using the Adam optimiser with a learning rate of 0.001.

From the layer table we can see the total parameters are over 2 million which are the weights I froze and the trainable parameters are just under eight thousand which are the new top layers I added so only 7,686 were trained and the rest of the parameters belong to the frozen MobileNetV2 base and were not changed.

```
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Model: "functional_1"
```

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
data_augmentation (Sequential)	(None, 224, 224, 3)	0
rescaling (Rescaling)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 6)	7,686

```
Total params: 2,265,670 (8.64 MB)
Trainable params: 7,686 (30.02 KB)
Non-trainable params: 2,257,984 (8.61 MB)
```

From the resulted architecture flow we can see how the data flows through each layer.

The input layer accepts images of shape 224 pixels wide, 224 pixels tall and 3 colour channels which are RGB.

The data augmentation layer implements random flips and rotations during training without changing image dimensions.

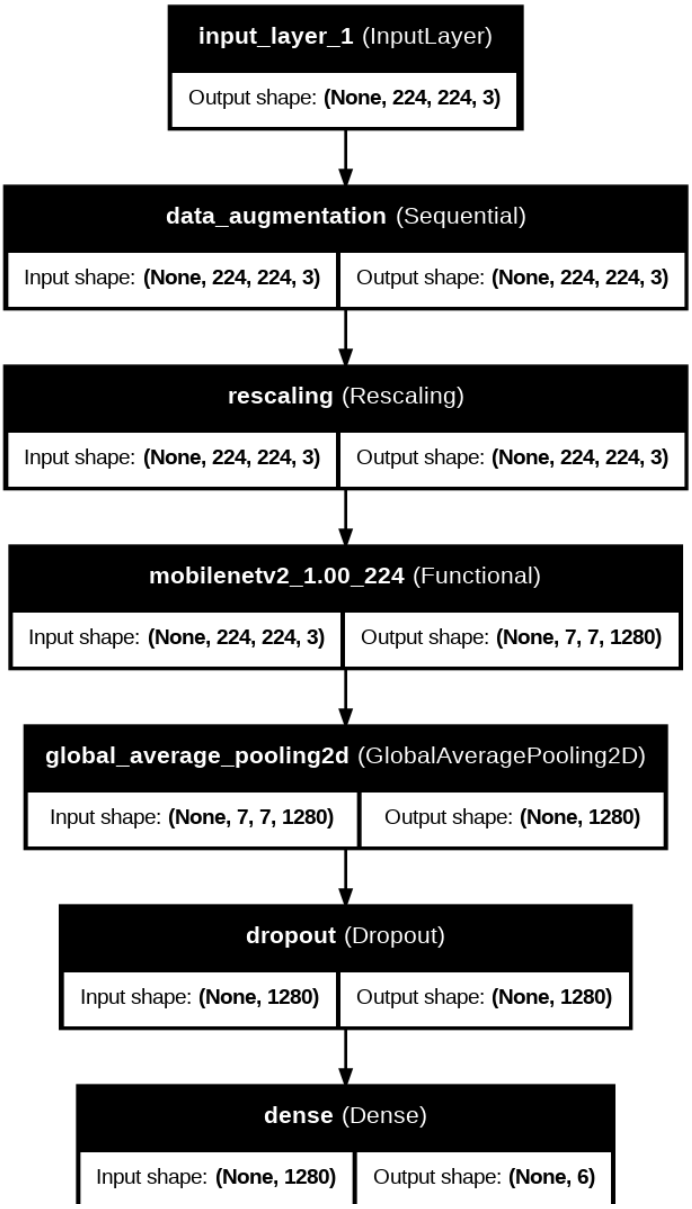
The rescaling layer normalises the pixel values for MobileNetV2.

The MobileNetV2 base then turns the image from 224x224x3 down to 7x7x1280.

GlobalAveragePooling2D changes the 7x7x1280 volume into a flat vector of 1280 values by getting an average across all the dimensions.

Dropout randomly zeroes 20% of those values during training to stop overfitting.

The dense layer with 6 outputs makes one score per class.

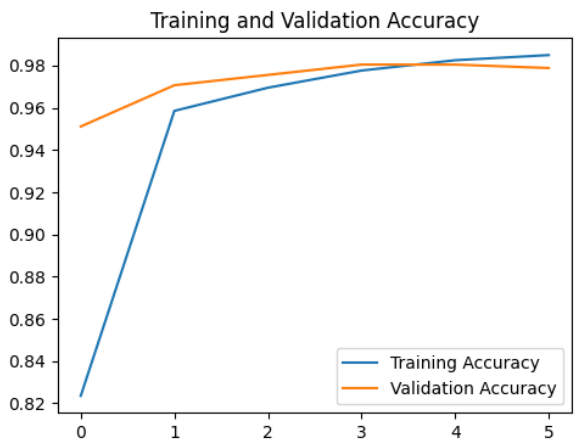


The model is then trained for 6 epochs, TensorFlow will evaluate the model on the validation set and it will show the training and validation accuracy and loss. These are plotted after training has completed which helps check if the model is learning properly and to see if it's overfitting.

We can see from the results below that training accuracy improved steadily from 82.4% in epoch 1 to 98.5% by epoch 6. The validation accuracy was higher than the training accuracy in the first epoch because dropout was active during training, then it was disabled during validation. The model had already achieved 97.1% validation accuracy by epoch 2 which shows the pretrained MobileNetV2 features carried over properly.

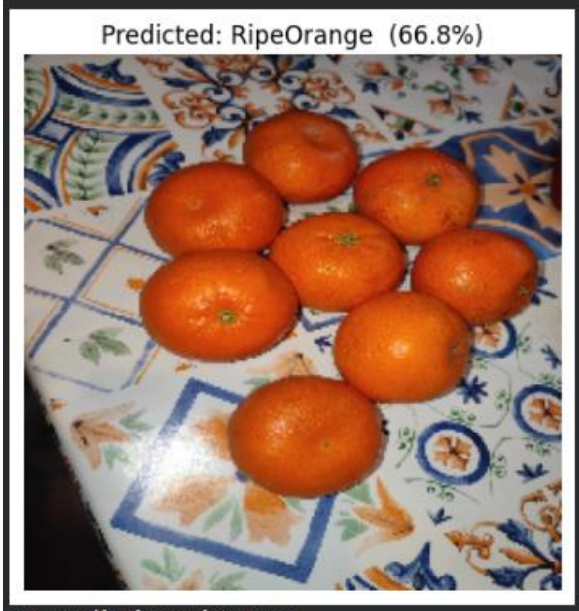
```
EPOCHS = 6

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
)

Epoch 1/6
77/77 ━━━━━━━━━━━━━━━━━━━━ 680s 9s/step - accuracy: 0.8236 - loss: 0.5370 - val_accuracy: 0.9511 - val_loss: 0.1656
Epoch 2/6
77/77 ━━━━━━━━━━━━━━━━━━━━ 138s 2s/step - accuracy: 0.9585 - loss: 0.1482 - val_accuracy: 0.9707 - val_loss: 0.1143
Epoch 3/6
77/77 ━━━━━━━━━━━━━━━━━━━━ 141s 2s/step - accuracy: 0.9695 - loss: 0.1054 - val_accuracy: 0.9756 - val_loss: 0.0894
Epoch 4/6
77/77 ━━━━━━━━━━━━━━━━━━━━ 139s 2s/step - accuracy: 0.9776 - loss: 0.0821 - val_accuracy: 0.9805 - val_loss: 0.0762
Epoch 5/6
77/77 ━━━━━━━━━━━━━━━━━━━━ 146s 2s/step - accuracy: 0.9825 - loss: 0.0683 - val_accuracy: 0.9805 - val_loss: 0.0713
Epoch 6/6
77/77 ━━━━━━━━━━━━━━━━━━━━ 143s 2s/step - accuracy: 0.9850 - loss: 0.0602 - val_accuracy: 0.9788 - val_loss: 0.0692
```

We can see from the plots that both accuracy lines stay close together and both the loss lines fall early on and level off. Since there is not a large gap between the training and validation lines, it shows the model is not overfitting to the training data.



I then tested the trained model on images it was not trained on to examine the accuracy; I took a photo using my phone and uploaded the image to my Google drive, the quality of the photo was not ideal, but the trained model provided the correct prediction.

```
# 1) Load and resize to match training
img = tf.keras.utils.load_img(img_path, target_size=IMG_SIZE)
x = tf.keras.utils.img_to_array(img)
x = np.expand_dims(x, axis=0)  # model expects (1, 224, 224, 3)

# 2) Predict
probs = model.predict(x, verbose=0)[0]
pred_idx = int(np.argmax(probs))
pred_label = class_names[pred_idx]
pred_conf = float(probs[pred_idx])

# 3) Display
plt.imshow(img)
plt.axis("off")
plt.title(f"Predicted: {pred_label}  ({pred_conf*100:.1f}%)")
plt.show()

print("Top prediction:", pred_label)
print("Confidence:", round(pred_conf * 100, 2), "%")
```




```
Top prediction: RipeOrange
Confidence: 66.8 %
```

Finally, the trained model was exported.

The model is saved in Keras format to a local output directory using “model.save”

The class names list is written to a plain text file “labels.txt” which shows all fruits and stages of ripeness available.

The model is then converted to TFLite format using “TFLiteConverter.from_keras_model”. The “DEFAULT” optimisation flag is applied which implements post training quantisation to reduce the models file size therefore, improving execution speed on the Raspberry Pi. The resulted “model.tflite” and “labels.txt” were transferred onto the Pi’s filesystem where the FastAPI server loads them upon startup.

```python
OUTPUT_DIR = "/content/fruit_ripeness_output"
os.makedirs(OUTPUT_DIR, exist_ok=True)

# 1) Save Keras model
keras_path = os.path.join(OUTPUT_DIR, "model.keras")
model.save(keras_path)
print("Saved Keras model to:", keras_path)

# 2) Save class labels
labels_path = os.path.join(OUTPUT_DIR, "labels.txt")
with open(labels_path, "w") as f:
    for name in class_names:
        f.write(name + "\n")
print("Saved labels to:", labels_path)

# 3) Convert to TFLite and optimize it
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()

tflite_path = os.path.join(OUTPUT_DIR, "model.tflite")
with open(tflite_path, "wb") as f:
    f.write(tflite_model)

print("Saved TFLite model to:", tflite_path)
```

```
Saved Keras model to: /content/fruit_ripeness_output/model.keras
Saved labels to: /content/fruit_ripeness_output/labels.txt
Saved artifact at '/tmp/tmpkju1oijh'. The following endpoints are available:
```

```
Saved TFLite model to: /content/fruit_ripeness_output/model.tflite
```

8 Testing

Mobile app tests below

Test	Steps	Expected	Actual	Pass/Fail
Navigate to capture	Tap new prediction on home screen	Capture screen opens	Opens Correctly	Pass
Phone camera permission	Press use phone camera	Permission prompt appears	Prompt shown	Pass
Phone camera prediction	Take a photo	Label and score shown	RipeBanana 99.2% shown	Pass
Pi camera prediction	Press use Pi camera	Label and score shown	RipeOrange 92.3% shown	Pass
Result card colour	Get an unripe result	Shows orange colour	Orange is shown	Pass
History screen loads	Navigate to History	Past predictions are shown	20 previous predictions shown	Pass
History auto refreshes	Make a prediction, navigate to history	New entry at the top	Latest entry visible	Pass
Error handling	Turn off Pi, try to get a prediction	Error message shown	Alert shown in the app	Pass

I created a separate test file to test the functionalities of the FastAPI server on the Pi, there are five tests.

Test 1 is used to check if the server is reporting itself as online by checking the /status endpoint.

```
# Test 1 - Status endpoint
# Sends a GET request to /status and checks the server
# responds with 200 OK and reports itself as online.
response = requests.get(f"{BASE_URL}/status")
assert response.status_code == 200
data = response.json()
assert data["status"] == "online"
print("PASS - /status returned online")
```

Test 2 checks the /history endpoint returns even if the prediction list is empty.

```
response = requests.get(f"{BASE_URL}/history")
assert response.status_code == 200
data = response.json()
assert isinstance(data, list)
print(f"PASS - /history returned a list with {len(data)} entries")
```

Test 3 sends an image file to the /predict endpoint and confirms the response contains a label and a confidence score between 0 and 1.

```
with open("/home/rory/fruitchecker/test_images/test.jpg", "rb") as f:
    response = requests.post(
        f"{BASE_URL}/predict",
        files={"file": ("test.jpg", f, "image/jpeg")}
    )
assert response.status_code == 200
data = response.json()
assert "label" in data
assert "score" in data
assert 0.0 <= data["score"] <= 1.0
print(f"PASS - /predict returned label='{data['label']}' score={data['score']}")
```

Test 4 tests the error handling by sending a request to /predict with no image attached and confirms the server returns an error instead of crashing.

```
response = requests.post(f"{BASE_URL}/predict")
assert response.status_code in [400, 422]
print(f"PASS - /predict correctly rejected empty request with {response.status_code}")
```

Test 5 is an integration test that makes a prediction and checks the /history endpoint to confirm the result was saved correctly by the server

```
with open("/home/rory/fruitchecker/test_images/test.jpg", "rb") as f:
    requests.post(f"{BASE_URL}/predict",files={"file": ("test.jpg", f, "image/jpeg")} )
response = requests.get(f"{BASE_URL}/history")
data = response.json()
assert len(data) > 0
latest = data[0]
assert "label" in latest
assert "score" in latest
assert "timestamp" in latest
print(f"PASS - history saved entry with label='{latest['label']}' score={latest['score']}")
```

```
Running API tests...

PASS - /status returned online
PASS - /history returned a list with 20 entries
PASS - /predict returned label='UnripeOrange' score=0.5873
PASS - /predict correctly rejected empty request with 422
PASS - history saved entry with label='UnripeOrange' score=0.5873

Done.
```

9 Problems faced and Solutions

In this section I will detail the problems I faced during this project and how I solved them, broken down by the area of the project they affected.

Raspberry Pi Setup

When I received the Pi, I had to flash the SD card to allow me to access the OS. I tried multiple times to set the Pi up through PowerShell locally and configure it, but I kept hitting the same problems. After flashing the SD card through Raspberry Pi imager, I inserted the SD card into the Pi and connected a monitor, keyboard and mouse directly to the Pi and finished configuring it in its own OS.

Pi Camera

Before integrating the camera with the prediction system, the Pi camera was tested through the terminal to ensure it was functional before testing with code.

Another issue I faced with the Pi camera was that it became unavailable when another process was already using it. This caused buffer queue and timeout errors. I had to find out which processes were using the camera's resources and stop them before I could use the camera for predictions. During camera testing, multiple camera devices were detected, which was reduced to one for simplicity and to avoid confusion.

Trained Model

For the machine learning model, I encountered several problems while building it. An earlier iteration of the Colab notebook required rerunning the notebook each time I wanted to test the model, which was inefficient and time consuming. To fix this I saved the trained model and exported it to TensorFlow Lite format, which allowed me to save the trained model in two files, model.tflite which contains the trained model and labels.txt which contains all the fruit and ripeness labels. I could then export these files to the Raspberry Pi's filesystem and deploy them there.

An issue I had in the Colab notebook was that OpenCV reads images in BGR colour order while the model expected RGB. This was fixed by converting the images from BGR to RGB before passing them into the model.

FastAPI Server on the Pi

The original plan was to use tflite-runtime to run the TensorFlow Lite model on the Pi, but I ran into compatibility issues with the Pi's Python environment. After doing research on workarounds for this issue, I switched to LiteRT through the ai_edge_litert package which allowed me to load the .tflite model successfully.

I was originally testing the trained model locally, but I made the decision to make the Pi more central to the project. I exported the trained model to the Pi, and all the image prediction logic now lives there. The UI runs locally while the predictions run from the Pi, and both connect via a ngrok HTTPS tunnel.

Image Prediction Accuracy

During testing the image prediction accuracy of the model, I noticed the model was not returning accurate predictions when capturing an image from either the phone camera or the Pi camera. After checking the model's output shape in the Pi's terminal, I noticed it had 7 output classes whereas my most recent labels file only had 6, which meant they were out of sync. I had to retrain the model and export the latest iteration so the model and labels files match. After correcting these bugs, I received consistent accurate predictions from the mobile app.

I found the server code had a double normalisation bug. The model already had a built-in Rescaling layer, but I had extra code that was dividing pixel values by 255 before passing the image to the model. I removed this section of code to fix the issue.

During further testing I noticed that when taking pictures of a bag of oranges in an orange net bag with mediocre lighting, no direct light source on the fruit and a dark or cluttered background, the model returned lower confidence scores. When I took the oranges out of the bag, used a white background, and used a torch for lighting, the confidence scores improved significantly. I received RipeOrange at 92.3% from the Pi camera and 87.5% from the phone camera. This highlighted how important consistent image quality is since my image dataset consists of studio-style images.

Mobile App

I originally planned on building a PyQt web app to display the predictions, but as I progressed through the Mobile App Development module, I realised the potential to use a mobile app to deploy the project. This approach would work better in a real-world

scenario, as the user will be carrying their phone in their pocket which makes it easier to take a picture of fruit and get a prediction anywhere.

For issues I ran into when building the mobile app, I initially had HTTP URLs hardcoded in the frontend which would not work on different networks. I solved this by setting up ngrok for the server on the Pi and updating the frontend to point at the ngrok URL, which allows communication over different networks.

I used an Android emulator to test the mobile app, and additional setup was required for this, such as installing SDKs and platform tools which were set up in Android Studio.

10 Ethics

While considering the ethics of this project, one of the concerns I have identified was the reliability of the model's predictions because if a user wanted to rely on the app to make the decision of whether the fruit is safe to eat, an inaccurate prediction could be dangerous for the user. I addressed this by displaying a confidence score with every result so the user can see how confident the model is with the prediction it gave.

The goal I had throughout the construction of this project was to have a product which is user friendly with the potential to assist users with certain visual impairments such as a form of colour blindness, I made the app simple to navigate through and to explore all its functionalities.

Another ethical factor that I took into consideration is image capture and storage. When a prediction is made, the image is encoded and stored on a history file on the Raspberry Pi and is currently not being exposed to an external server but if the system was to be deployed more widely, it would be important to inform users that their images are being stored and to give them an option to delete their history.

11 AI Use

Throughout the development of the project, I have made use of AI tools, primarily ChatGPT, to help with learning and problem solving. I used AI as a help for research, debugging, and explanations. This project was my idea, and I did not use AI to complete the project for me.

When creating the Google Colab notebook, I used AI to help me better understand how I could reach the goal of training a model and which strategy would be efficient and doable within a final year project. It helped me build a basic picture of the steps involved and assisted me in debugging through the notebook, which helped my understanding of how the model training process works.

For the mobile app, the fundamental code and knowledge came from the Mobile App Development module, and I used AI on top of this to aid with debugging issues and researching how to solve problems within the project and how to structure the project for a fruit ripeness app.

I used AI to give me a greater understanding of how I could use the Raspberry Pi as a backend for the mobile app and to explore the best options I could use to use the Pi as the backend and how I could setup the basics in the file. I used AI to assist me with the debugging process for getting the server functional also such as making the switch from tflite-runtime to LiteRT.

AI was used for researching and gaining an understanding of what tests would be necessary to make to test the API server on the Pi, which helped me understand how to structure the tests.

AI was used for general research such as examining which public HTTPS tunnelling tools would be compatible with my project, debugging Raspberry Pi hardware issues such as flashing the OS, camera connectivity issues and researching into making the poster, report and to help me better understand some of the logic in the project.

I also used AI to assist with the report such as small spelling mistakes, formatting errors, and editing, I wrote the report myself.

12 Conclusion

The final year project allows the student to combine their knowledge gained from their time in college and showcase it in a project. I explored many different areas which were new to me and had to learn some aspects from scratch, but I was able to apply the problem solving, debugging, and troubleshooting skills that I have broadened to this project.

The system I built has 3 parts, a trained MobileNetV2 image classification model, a FastAPI server running on the Raspberry Pi and a React Native mobile app. The model achieved a validation accuracy of 97.88% after 6 epochs of training in Google Colab. I have gained an understanding of multiple different technologies such as training a machine learning model, setting up, and configuring a Raspberry Pi, building a REST API in Python, and developing a mobile app in React Native. My project management skills were improved using OneNote and Jira where I documented progress and logged anything project related.

If I was going to develop this project further, I would investigate training the model using more fruits and more varied images such as images with varied lighting conditions and backgrounds to improve the model's real-world performance. Another useful addition would be to add the ability for the user to delete predictions from the history and export the results as this would improve the overall user experience. Another addition would be adding a login page with facial recognition that would act as a password and find a way to deploy the app so it could be downloaded and used without having to run and setup different services.

13 References

[1] Python Software Foundation, "What is Python?," [Online].

Available: https://www.python.org/doc/essays/blurb/.

[Accessed April 2026].

[2] Uvicorn, "Uvicorn - An ASGI web server implementation for Python," [Online].
Available: https://www.uvicorn.org/.

[Accessed April 2026].

[3] FastAPI, "FastAPI Documentation," [Online].

Available: https://fastapi.tiangolo.com/.

[Accessed April 2026].

[4] OpenCV, "OpenCV Documentation," [Online].

Available: https://docs.opencv.org/. [Accessed April 2026].

[5] NumPy, "NumPy Documentation," [Online].

Available: https://numpy.org/doc/. [Accessed April 2026].

[6] Raspberry Pi Foundation, "Picamera2 Manual," [Online].

Available: https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf.

[Accessed April 2026].

[7] Google, "Google Colaboratory," [Online].

Available: https://colab.research.google.com/.

[Accessed April 2026].

[8] TensorFlow, "TensorFlow Documentation," [Online].

Available: https://www.tensorflow.org/.

[Accessed April 2026].

[9] TensorFlow, "Keras Documentation," [Online].

Available: https://www.tensorflow.org/guide/keras.

[Accessed April 2026].

[10] TensorFlow, "MobileNetV2," [Online].

Available: https://www.tensorflow.org/api_docs/python/tf/keras/applications/MobileNetV2. [Accessed April 2026].

[11] React Native, "React Native Documentation," [Online].

Available: https://reactnative.dev/.

[Accessed April 2026].

[12] Expo, "Expo Documentation," [Online].

Available: https://docs.expo.dev/.

[Accessed April 2026].

[13] ngrok, "ngrok Documentation," [Online].

Available: https://ngrok.com/docs.

[Accessed April 2026].

[14] Google, "LiteRT Documentation," [Online].

Available: https://ai.google.dev/edge/litert.

[Accessed April 2026].

[15] RealVNC, "VNC Connect Documentation," [Online].

Available: https://www.realvnc.com/en/connect/docs/.

[Accessed April 2026].

[16] Atlassian, "Jira Software Documentation," [Online].

Available: https://www.atlassian.com/software/jira/guides.

[Accessed April 2026].

[17] leftin, "Fruit Ripeness: Unripe, Ripe, and Rotten," Kaggle, [Online].

Available: https://www.kaggle.com/datasets/leftin/fruit-ripeness-unripe-ripe-and-rotten.

[Accessed September 2025].